Assay Document Audit & Improvement Recommendations

Audience: Engineering leadership, for review before scheduling any follow-up work. **Author:** Engineering, with Claude Code **Date:** 2026-04-16 **Status:** Findings + recommendations. The write-path parity fix (Section 3) has already been implemented. Everything in Section 4 is proposed; nothing has been changed.

---

1. TL;DR

- The `assay_definitions` collection currently holds **238 assays**, all of them imported from the prior system. **Zero** assays have been created in BIMS (we verified — every doc carries the legacy markers).
- However, the BIMS creation paths were writing a **different document shape** than the imported ones. If an operator had successfully created an assay through BIMS, the resulting document would have been missing the `BCODE` instruction block, would have had a nanoid-style `_id` instead of the legacy `A########` format, and would have stored instructions under `metadata.instructions` rather than `BCODE.code`. Any downstream consumer (firmware, test-execution code) that reads `BCODE` would have found nothing on BIMS-created docs.
- **Immediate fix (now done):** BIMS write paths now mint legacy-format IDs and populate `BCODE` in the legacy shape. A new assay created today through BIMS will match the imported documents structurally.
- **Still open (recommendations in this report):** the collection has four other shape inconsistencies, a dead-code area in the Mongoose schema, a broken unique index declaration, and a read-path gap where the detail/edit UI cannot display legacy instructions.

---

2. What's actually in mongo (current state)

238 assay documents, clustered into four distinct top-level key signatures:

#	Count	Distinctive keys
1	206	baseline legacy shape — `BCODE` (object), `hidden`, `protected`
2	3	adds clinical metadata: `cartridgeLife`, `diluent`, `functions`, `lotLife`, `matrix`, `version`, plus `useSingleCost`; here `BCODE` is a bare array, not `{deviceParams, code}`
3	5	sig #1 + `useSingleCost`
4	24	sig #3 + `lockedBy: {}` (empty object)

Universal markers on all 238:

- `_id` of form `A` + 7 uppercase hex chars (e.g. `A00E895F`, `AC93ACD3`)
- `BCODE` (uppercase) — holds the instruction program. Usually `{ deviceParams: {...}, code: [...] }`; the 3 clinical docs (signature #2) use a bare array
- `hidden: true`, `protected: true`
- `skuCode: null` for **every single doc**
- `reagents: []` empty on all docs
- `versionHistory: []` empty on all docs
- No `bcode` (lowercase Buffer), `bcodeLength`, `checksum`, or `metadata` on any doc

Sample of the legacy `BCODE` shape (signature #1)

```
{
  "deviceParams": {
    "delayBetweenSensorReadings": 100,
    "integrationTime": 128,
    "gain": 0,
    "ledPower": 300
  },
  "code": [
    { "command": "Start Test", "params": {} },
    { "command": "Move Microns", "params": { "comment": "...", "microns": 2850,
"step_delay_us": 5000 } },
    { "command": "Delay",        "params": { "comment": "...", "delay_ms": 150000 } },
    { "command": "Oscillate Stage", "params": { "comment": "...", "microns": 2250,
"step_delay_us": 350, "cycles": 178 } },
    { "command": "Repeat", "count": 44, "code": [
      { "command": "Read Sensor", "params": { "channel": 0, "gain": 7, "step": 499, "time":
49 } },
      { "command": "Move Microns", "params": { "microns": 100, "step_delay_us": 350 } }
    ] },
    { "command": "Finish Test", "params": {} }
  ]
}
```

Note: commands are **Title Case strings** ("Start Test", "Move Microns"); params are **objects with named keys** (`delay_ms` , `microns` , `step_delay_us` , etc.); `Repeat` uses a top-level `count` plus an inline nested `code` array (no explicit `Repeat End`).

---

3. What BIMS was writing (before the fix)

The schema in `src/lib/server/db/models/assay-definition.ts` declares fields that **no legacy doc has and no legacy doc reader needs**: `bcode: Buffer` , `bcodeLength: Number` , `checksum: Number` , `metadata: Mixed` , `skuCode: {required: true, unique: true}` .

The creation paths wrote against that schema:

Path	Problems with the write
`src/routes/assays/new/+page.server.ts`	`_id` from `generateId()` (nanoid), instructions into `metadata.instructions`, no `BCODE`, no `hidden/protected`, skuCode auto-set to `ASSAY-<timestamp-b36>`
`src/routes/assays/import/+page.server.ts`	`_id` always freshly-generated nanoid; did not preserve legacy `BCODE`, `hidden`, `protected` if present in import file
`src/routes/assays/[assayId]/+page.server.ts` — `duplicate`	`_id` via `generateId()` (nanoid). Spread-copy of `...rest` would have carried `BCODE` over if the source had it, but Mongoose strict mode strips unknown fields on write, so the copy would drop `BCODE`
`src/routes/assays/+page.server.ts` — `duplicate` (list page)	`_id` via `generateId()`, forced suffix on `skuCode`, no `BCODE`, no legacy markers

`src/routes/manufacturing/reagent-filling/settings/+page.server.ts` — `createAssayType`	`_id` default (nanoid). No `BCODE` (expected — manufacturing-side assay types don't have a protocol)
`src/routes/cartridge-admin/sku-management/+page.server.ts`	`_id` via `generateId()` (nanoid)

The `duplicate` spread-copy issue above is the most subtle: even though the code tried to preserve fields with `...rest`, Mongoose's default strict mode strips fields that aren't declared in the schema, so `BCODE` was silently dropped on every duplicate.

The fix that's now in place

1. **New utility** at `src/lib/server/assay-legacy-shape.ts` with:
   - `generateLegacyAssayId()` — returns `A` + 7 hex chars, collision-checked against the existing collection.
   - `toLegacyBcode(instructions, deviceParams?)` — translates the UI's instruction shape (UPPERCASE_TYPE + number[] params) into legacy `BCODE.code` format (Title Case command + named-object params). Covers the confirmed opcodes (`START_TEST` , `END_TEST` , `DELAY` , `MOVE_MICRONS` , `OSCILLATE` , `SENSOR_READING` , `REPEAT_BEGIN`); emits a best-effort `{command, params: {raw_type, raw_params}}` for unconfirmed ones (see Section 5).
2. **Schema** now declares `BCODE: Mixed` , `hidden: Boolean` , `protected: Boolean` , and drops `skuCode` 's `required/unique` so it can be null like legacy docs. The dead `bcode` / `bcodeLength` / `checksum` fields were left in place for now (no reader removal yet — belongs to a follow-up cleanup).
3. **Every create path** listed above now uses `generateLegacyAssayId()` and writes `hidden: true, protected: true` . The protocol-owning paths (`new` , `import` , the two duplicates) write `BCODE` . The manufacturing-side paths (`reagent-filling/settings` , `sku-management`) don't write `BCODE` since they have no protocol data — but they still produce `A########` IDs and carry the legacy markers.

Verification steps (suggested)

1. Create a new assay through BIMS. Confirm in mongo that the new doc has `_id` matching `^A[0-9A-F]{7}$` , has a `BCODE` object, has `hidden: true` and `protected: true` , and lacks `metadata` .
2. Duplicate an imported assay. Confirm the copy preserves `BCODE` and has a fresh `A########` `_id` .
3. Run the dry-run audit script (`scripts/find-bims-assays.ts`) a day after users start creating — it will flag any doc deviating from legacy markers.

---

4. Recommended follow-up improvements (NOT yet implemented)

Ordered by impact.

4.1 Fix the detail/edit pages so legacy instructions display

Problem. `src/routes/assays/[assayId]/+page.server.ts:36` reads `assay.metadata?.instructions` to populate the detail-page instruction list. Legacy docs have instructions in `BCODE.code` , not `metadata.instructions` , so the detail page currently shows **no instructions for any of the 238 existing assays**. Same for the edit page fallback at `edit/+page.server.ts:16` .

Recommendation. Add a reverse translator (`fromLegacyBcode(BCODE)` → UI shape) as a counterpart to the `toLegacyBcode` we added. The load functions should prefer `BCODE.code` if present, translate it back to the UI's

opcode shape, and fall back to `metadata.instructions` only for any stragglers. This lets the UI edit imported assays.

Effort. Medium. The translator is symmetric to the one we wrote. Needs careful handling of nested `Repeat` blocks.

Risk. Low. Pure additive change on the read path; doesn't touch mongo.

4.2 Remove the dead `bcode` / `bcodeLength` / `checksum` fields

Problem. The schema declares `bcode: Buffer`, `bcodeLength: Number`, `checksum: Number`. **Zero** docs in mongo have any of these fields, and **no code path writes them**. The detail page reads them (`[assayId]/+page.server.ts:26-33`) but always returns null. They are fossils.

Recommendation. Drop the three fields from the schema. Remove the read sites and the bcodeLength column in the assay list. This simplifies mental model and removes a misleading column from the UI.

Effort. Small. ~4 files touched, no data migration.

Risk. Very low — deleting unused code.

4.3 Clarify `BCODE` as a single well-defined shape

Problem. Signature #2 (3 clinical assays) stores `BCODE` as a bare array, while the other 235 store it as `{deviceParams, code: [...]}`. Read code that assumes one will crash on the other.

Recommendation. Migrate the 3 array-form docs to object form (`{ deviceParams: DEFAULT, code: <existing array> }`). Lock down a single shape in code (a TypeScript interface in `assay-legacy-shape.ts`) and reject writes that don't match.

Effort. Small. 3-doc migration + a runtime validator.

Risk. Low — the 3 clinical assays (`CRP Quantitative`, `IL-6 Quantitative`, `TNF-alpha Semi-Quantitative`) don't appear to be consumed by firmware today, so normalizing them to match the rest is safe. Worth confirming they're not in production use first.

4.4 Re-introduce `skuCode` uniqueness as a partial index

Problem. Schema originally declared `skuCode: { required: true, unique: true }` but all 238 legacy docs have `skuCode: null`. Mongo's unique index treats `null` as a value, so only one doc with null was technically allowed — the index either never built or was deferred. We relaxed the field in the fix so BIMS could write null; now there's no uniqueness guard.

Recommendation. Add a partial unique index: `{ skuCode: 1 }` unique, with filter `skuCode: { $type: 'string' }`. That lets nulls coexist (as they do now) while still preventing two BIMS-created assays from colliding on the same SKU.

Effort. Small — one-line migration script to create the index. But: **any existing duplicates among non-null skuCodes would block index creation**, so the migration should first scan for duplicates and report.

Risk. Low.

4.5 Populate `reagents` on imported assays

Problem. All 238 docs have `reagents: []`. The schema declares a rich reagent subdocument structure (with `subComponents`, `wellPosition`, etc.) that the reagent-filling workflow depends on. Currently, cartridges created

during reagent filling attach a reagent snapshot from `AssayDefinition.reagents` — but that snapshot is always empty for legacy assays.

Recommendation. Understand where reagent composition lives for the 238 legacy assays (possibly in the prior system's database, possibly on paper). Add a one-time import flow to populate `reagents[]` on the docs that need it. Going forward, the assay detail page already supports adding/editing reagents in-place.

Effort. Large. Depends heavily on whether source data exists in a structured form.

Risk. Medium — scope is unclear without knowing what the source data looks like.

4.6 Lock down the instruction vocabulary with the firmware team

Problem. The UI's `BcodeEditor` emits 11 opcode types (`START_TEST` , `DELAY` , `MOVE_MICRONS` , `OSCILLATE` , `SET_SENSOR_PARAMS` , `BASELINE_SCANS` , `TEST_SCANS` , `SENSOR_READING` , `CONTINUOUS_SCANS` , `REPEAT_BEGIN` , `END_TEST`). Legacy `BCODE.code` uses a Title-Case vocabulary like "Start Test", "Move Microns", "Delay", "Oscillate Stage", "Read Sensor", "Repeat", "Finish Test". Only **6** UI opcodes have a confirmed legacy equivalent. For the other 5 (`SET_SENSOR_PARAMS` , `BASELINE_SCANS` , `TEST_SCANS` , `CONTINUOUS_SCANS` , `REPEAT_END`), the translator currently emits a best-effort placeholder that firmware may or may not recognize.

Recommendation. Meeting with firmware owners to (a) confirm the canonical command vocabulary, (b) map the 5 unconfirmed opcodes to legacy equivalents or add new ones, and (c) document the final list in `assay-legacy-shape.ts` . Until this happens, any new assay that uses one of the unconfirmed opcodes is a firmware risk.

Effort. 1 meeting + small code change.

Risk. Low if done before users start authoring new protocols; medium if deferred and assays start getting created with ambiguous opcodes.

4.7 Distinguish "protocol-bearing" vs. "manufacturing-type" assays

Problem. The `assay_definitions` collection is serving two very different use cases:

1. **Protocol records** — the 238 imported experimental protocols, each with a `BCODE` program, no reagent composition, usually `hidden: true` .
2. **Manufacturing SKU types** — assay "families" created through `cartridge-admin/sku-management` and `reagent-filling/settings` to categorize what's being filled on the manufacturing line. These need reagent composition but don't need a protocol.

Mixing them means the schema has to support both, which is why there are so many optional fields and why queries sometimes confuse the two.

Recommendation. Evaluate splitting into two collections (`assay_protocols` and `assay_sku_types`), or at minimum adding a `kind: 'protocol' | 'sku_type'` discriminator and filtering every list page accordingly. Low-urgency, but worth discussing before the manufacturing side starts creating lots of records.

Effort. Medium if split; small if just a discriminator field.

Risk. Medium — touches several pages; worth careful UX review.

4.8 Remove `versionHistory` as a stub, or wire it up

Problem. The schema declares `versionHistory: [...]` with rich fields (`previousBcode` , `previousName` , etc.). The `edit` action appends an entry on each save. The UI displays it via `VersionHistory.svelte` . But: **zero of the 238 legacy docs have a non-empty `versionHistory`** , and the edit action doesn't actually snapshot `BCODE` — only the top-level metadata fields. So what gets recorded is partial at best.

Recommendation. Pick one: (a) wire it up properly — snapshot `BCODE` into each version entry, so edits are fully auditable; or (b) drop it in favor of the existing `AuditLog` collection, which already captures every mutation across the app and is the intended audit mechanism.

Option (b) seems cleaner (one audit surface, not two).

Effort. Small either way.

Risk. Low.

4.9 Decide whether `hidden` and `protected` should default to `false` for BIMS-created assays

Problem. The immediate fix sets `hidden: true, protected: true` on every new BIMS-created doc to match legacy uniformly. But these fields probably mean "hide from normal listing" and "don't allow deletion" — defaults that may not be right for assays users are actively authoring.

Recommendation. Confirm the intended semantics with stakeholders. If `hidden` means "archive / don't surface," new assays should default to `false`. If it's just a legacy artifact, drop it entirely.

Effort. Small — one-line change once the decision is made.

Risk. Low.

---

5. Known translator uncertainties (flag for firmware review)

The `toLegacyBcode()` translator in `src/lib/server/assay-legacy-shape.ts` handles 6 opcodes with high confidence (based on observed legacy docs) and falls back for the rest:

UI opcode	Confirmed legacy command?	Notes
`START_TEST`	✅ `"Start Test"`	1:1
`END_TEST`	✅ `"Finish Test"`	1:1
`DELAY`	✅ `"Delay"` with `{delay_ms}`	1:1
`MOVE_MICRONS`	✅ `"Move Microns"` with `{microns, step_delay_us}`	1:1
`OSCILLATE`	✅ `"Oscillate Stage"` with `{microns, step_delay_us, cycles}`	1:1
`SENSOR_READING`	✅ `"Read Sensor"` with `{channel, gain, step, time}`	UI's `integration` maps to legacy's `time`
`REPEAT_BEGIN` + inline `code`	✅ `"Repeat"` with top-level `count` + inline `code`	1:1, implicit end
`REPEAT_END`	N/A	Skipped; end is implicit in legacy
`SET_SENSOR_PARAMS`	❓	No direct legacy match seen. Possibly belongs in `deviceParams` instead of `code`.

`BASELINE_SCANS`	❓	Legacy expresses this as a `Repeat` block around `Read Sensor` + `Move Microns`.
`TEST_SCANS`	❓	Same as BASELINE_SCANS.
`CONTINUOUS_SCANS`	❓	Unclear — no clean legacy analog observed.

Anything marked ❓ currently gets emitted as `{ command: TitleCase(opcode), params: { raw_type, raw_params: [...] } }` so the data survives round-trip but firmware will need to be taught about it. See Section 4.6.

---

6. Summary of what's in flight

Item	Status
Zero data migration needed — there are no BIMS-created docs in prod today	Verified
BIMS write paths produce legacy-shape docs going forward	**Done**
Detail/edit pages display legacy instructions	Open — Section 4.1
Dead `bcode`/`bcodeLength`/`checksum` cleanup	Open — Section 4.2
Normalize BCODE shape (object vs. array)	Open — Section 4.3
Re-introduce partial unique index on skuCode	Open — Section 4.4
Backfill `reagents[]` on legacy docs	Open — Section 4.5
Firmware vocabulary lock-down	Open — Section 4.6
Split or discriminate assay kinds	Open — Section 4.7
Wire up (or remove) versionHistory	Open — Section 4.8
Revisit hidden/protected defaults	Open — Section 4.9